

Week - 04 - Exceptions, Dataflow\Lecture_7_code_3.py

```
1  '''
2  A functional (data flow driven approach) to write code. Note how we call
3  functions in a chain at line 44
4  '''
5  scores = [
6      ["12", "15", "missing"],
7      ["8", "10", "9"],
8      ["18", "absent", "13"],
9  ]
10
11 def gen_score(score_matrix):
12     for raw_record in score_matrix:
13         yield raw_record
14
15
16 def process_record(raw_record):
17     record = []
18     for element in raw_record:
19         try:
20             record.append(int(element))
21
22         except ValueError as e:
23             record.append(0)
24
25     return record
26
27 def sum_points(record):
28     return sum(record)
29
30 def assign_grades(total_marks):
31     if total_marks > 30:
32         return 5
33     elif total_marks > 25:
34         return 4
35     elif total_marks > 20:
36         return 3
37     else:
38         return 'U'
39
40 for raw_record in gen_score(scores):
41     # this line is written in a functional style that exhibits how
42     # data flows inside our system. That is a more natural match
43     # to a data flow diagram
44     print(assign_grades(sum_points(process_record(raw_record))))
45
46
47     # Down here is more imperative style programming, which exhibits how
48     # control flows inside of our system.
49     # Nonetheless, data is also present, but we make programm in a more
50     # control oriented way
51
```

```
52     # # print(raw_record)
53     # record = (process_record(raw_record))
54     # total_marks = sum_points(record)
55     # # print(total_marks)
56     # grade = assign_grades(total_marks)
57     # print(grade)
58
```